MagFluid3S

User Guide
(v1.4.0)

Juan Zapata
PhD Student

Grupo de Magnetismo y Simulación
Facultad de Ciencas Exactas y Naturales
Universidad de Antioquia
2018-2024

Laboratorio de Bajas Temperaturas
Centro Atómico de Bariloche
2024-2026

Table of Contents

List of Figures

List of Tables

List of Scripts

Preface

This project began in mid-2018, while I was finishing my Physics degree at the University of Antioquia. It was born as an initiative to study the thermodynamic and structural properties of magnetic fluids based on nanoparticles. At first, a two-dimensional study was carried out using Monte Carlo methods in an energy minimization process with the Metropolis algorithm. Then, at the beginning of 2020, during my Master's studies in Physics, the idea arose to extend the system to three dimensions and focus on hysteretic properties. The Monte Carlo approach was continued, but in January 2021 a Molecular Dynamics scheme was incorporated (the only one still in use), solving the Landau–Lifshitz–Gilbert (LLG) equation, as this allowed the introduction of a time variable. Some experimental results were successfully reproduced, and the implementation was entirely satisfactory.

However, the project presented many opportunities for improvement, in my view. Although I had completed my master's studies in mid-2022, I continued to dedicate time to the project in my spare time, as there were several aspects that did not convince me: the way data were generated, read, and stored; ambiguous or redundant structures that slowed down the simulation; an impractical debugging and improvement process; and the absence of an input file, something essential in this type of simulation. In addition, the code had organizational and stylistic problems: functions were poorly structured, variables were declared inconsistently, and syntax and style did not follow good programming practices. It lacked documentation and, in general, although functional, the program was quite limited.

Improving the code and making it professional became a major personal challenge. What motivated me most was the possibility of further developing my programming skills and the idea of integrating artificial intelligence into the project. After continuous improvements over approximately two years (during free weekends or vacations), around 2024 the idea emerged to rigorously document the code and write this manual, something I had never done before. I began my Ph.D. studies in Physics in mid-2024, focusing on superconductivity (a different topic); however, I continued improving the code and started writing this manual. Ultimately, it is a project that has required me to grow both in the fields of physics and programming, as well as in scientific writing.

The next step, and a great motivation, is that after completing this document, there is the latent possibility of legally registering the code with the relevant authorities, having a finalized product, and being recognized as its intellectual and material author. The mere thought of it fills me with pride.

To be honest, I am not sure how far this project might go, as there are still some features to be incorporated: the inclusion of particle–particle interactions and translational dynamics, which would imply revisiting the study of structural properties and adding a visualization module for configurations (what I call "microstates"). It is a long-term plan that will not be easy and will require much more learning. Those who work in this field know well that adding short- and long-range interactions represents an entirely new challenge, both in programming and efficiency, as well as in maintaining the coherence of the results. I already have some preliminary ideas and other novel ones based on AI that I would like to implement.

I hope this work may one day be useful to someone. It already is for me, since the personal and professional growth it has provided has no comparison, as it has given me ideas to develop other, more ambitious projects. I sincerely hope that the reader enjoys reading this manual and finds it very helpful.

My most sincere regards,

10/15/2025
Juan Zapata
Author

Motivation

MagFluid3S (Magnetic FluidS Stochastic Simulations) is a free and open-source software developed for the simulation of dynamic and thermodynamic properties of three-dimensional magnetic nanoparticle systems in a viscous medium. The framework allows users to model, initialize, execute, and post-process different types of simulations, including Microstates (observation of the dynamics of magnetic moment and easy-axis), MvsH (magnetization versus magnetic field, hysteresis loops), and MvsT (magnetization versus temperature, ZFC and FC protocols). The code supports flexible configuration of particle size distributions, material parameters, and initial magnetic states, enabling detailed studies of the magnetic response of nanoparticle ensembles under a wide range of experimental conditions.

Implemented in both Python and Fortran, the framework combines the versatility of Python for workflow control and data processing with the computational efficiency of Fortran–OpenMP for the numerical solvers. This hybrid design ensures both performance and usability. Particular emphasis is placed on reproducibility and extensibility, allowing users to modify, expand, and automate simulations to address new research challenges, especially in the fields of nanomagnetism and magnetic fluids.

Part I

Theoretical Background

CHAPTER 1

Physical Modeling

1.1. Introduction

In this chapter, the physical model implemented in the **MagFluid3S** framework is described, including the underlying assumptions, the equations governing the system, and the observables characterizing the behavior of nanoparticle-based magnetic fluids. These concepts form the basis for the stochastic simulations.

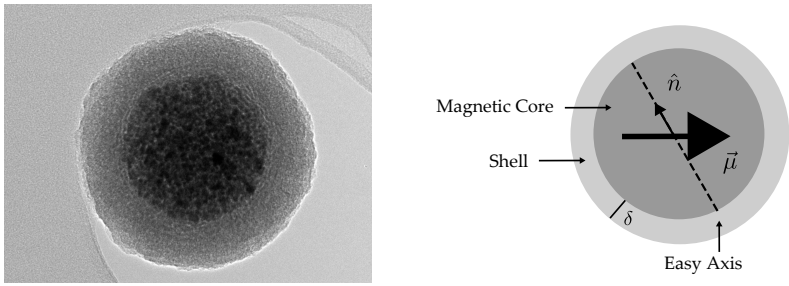
Section 1.2 describes the main *assumptions* considered in the model: the structural and magnetic properties of the nanoparticles, the role of thermal fluctuations, the potential energies determining their behavior, and the types of rotational motion involved (Néel and Brownian). Translational motion and interparticle interactions are neglected, and the particles are treated as a dilute suspension, where the dynamics is governed by rotational degrees of freedom.

Section 1.3 introduces the *dynamics* that describe the time evolution of the magnetic moments and the anisotropy axes. The formulation combines the Landau–Lifshitz–Gilbert (LLG) equation with a rotational torque equation and includes stochastic contributions representing thermal agitation, consistent with the fluctuation–dissipation theorem.

Section 1.4 discusses the temperature dependence of *viscosity*, described via a model based on the Vogel–Fulcher–Tammann (VFT) equation, providing a realistic representation of the solvent behavior in both solid and liquid phases.

1.2. Assumptions

The magnetic fluid under consideration consists of N magnetic nanoparticles dispersed in a non-magnetic viscous medium (water). For modeling purposes, only their essential magnetic and structural features are considered. Fig. 1.1 shows both a TEM image and a schematic representation of the particle structure. The following assumptions apply:



(a) TEM image [1]. (b) Schematic representation [2].

Figure 1.1. Nanoparticle structure.

Particles:

- Modeled as rigid spheres with radius R_m and volume Ω_m.
- Each particle is a single-domain magnetic core with magnetic moment:

$$\vec{\mu} = M_S \Omega_m \hat{m}, \tag{1.1}$$

  with M_S its saturation magnetization and $\hat{m}$ its orientation.
- Each magnetic core possesses a single anisotropy axis $\hat{n}$.
- Each magnetic core is coated with a non-magnetic shell of thickness δ.
- Both R_m and δ follow a log-normal size distribution:

$$\rho(x) = \frac{1}{\sqrt{2\pi} x \sigma_x} \exp\left[-\frac{(\ln(x) - \ln(x_0))^2}{2(\sigma_x)^2} \right], \tag{1.2}$$

with x_0 the mean value of the set $\{x_i\}_{i=1}^{N}$, σ_x the standard deviation of $\{\ln(x_i)\}_{i=1}^{N}$, and $x = \{R_m, \delta\}$. In the monodisperse limit, i.e., $\sigma_x = 0$, the distribution reduces to a Dirac delta function: $\rho(x) = \delta(x - x_0)$.

- Each particle has an overall radius and volume:

$$R_p = R_m + \delta, \tag{1.3}$$

$$\Omega_p = \Omega_m \left(1 + \frac{\delta}{R_m}\right)^3. \tag{1.4}$$

Thermal Fluctuations:

- Modeled as white noise (Sec. 2.4).
- Fluctuations do not alter the shape or size of the particles.
- Fluctuations do not change M_S (below the Curie-Weiss temperature).
- Fluctuations can change the orientation of $\hat{m}$ and $\hat{n}$ (Sec. 1.3).

Potential Energies:

- Zeeman:

$$U_Z = -\mu_0 \vec{\mu} \cdot \vec{H}, \tag{1.5}$$

with μ_0 the vacuum magnetic permeability and $\vec{H}$ an external magnetic field.

- Uniaxial Anisotropy:

$$U_A = -K_{eff} \Omega_m (\hat{m} \cdot \hat{n})^2, \tag{1.6}$$

with K_{eff} the effective anisotropy constant (including magnetocrystalline, surface, interfacial, and other contributions).

- Pairwise interactions (e.g., Lennard-Jones or dipole–dipole) are neglected.

Rotations and Translations:

- Néel rotation (Eq. 1.7).
- Brownian rotation (Eq. 1.8).
- Translational motion is neglected (will be implemented in a future version).

In this model, particle shape variability is ignored, assuming rigid-sphere geometry. Mass variations are neglected, as they do not affect the properties under consideration (Sec. 2.5). Finally, the suspension is considered highly diluted, which prevents significant interparticle interactions and agglomeration.

1.3. Dynamics

The time evolution of the magnetic moment orientation and the easy axis is described by the Landau–Lifshitz–Gilbert (LLG) equation for $\hat{m}$, and by a torque equation, derived from variational principles, for $\hat{n}$:

$$\frac{d\hat{m}}{dt} = -\frac{\gamma}{1+\alpha^2}\left[\hat{m}\times\vec{H}_{eff} + \alpha\ \hat{m}\times(\hat{m}\times\vec{H}_{eff})\right], \tag{1.7}$$

$$\frac{d\hat{n}}{dt} = -\frac{1}{\zeta}\ \hat{n}\times\vec{\theta}_{eff}, \tag{1.8}$$

with γ the electronic gyromagnetic ratio and α the damping parameter. The drag coefficient is given by $\zeta = 6\eta\Omega_p$, corresponding to a spherical particle of hydrodynamic volume Ω_p immersed in a fluid with viscosity η. The effective field $\vec{H}_{\text{eff}}$ and the corresponding effective torque $\vec{\theta}_{\text{eff}}$ are defined as:

$$\vec{H}_{eff} \equiv -\frac{1}{\mu_0\mu}\frac{\partial U}{\partial\hat{m}} + \vec{H}_{th} = \vec{H} + H_K(\hat{m}\cdot\hat{n})\hat{n} + \vec{H}_{th}, \tag{1.9}$$

$$\vec{\theta}_{eff} \equiv \hat{n}\times -\frac{\partial U}{\partial\hat{n}} + \vec{\theta}_{th} = -2K_{eff}\Omega_m(\hat{m}\cdot\hat{n})(\hat{m}\times\hat{n}) + \vec{\theta}_{th}, \tag{1.10}$$

with $U = U(\hat{m}, \hat{n}, t)$ the total potential energy.

In Eq. 1.9, the first term corresponds to the external magnetic field, whereas the second term represents the anisotropy field, with $H_K = \frac{2K_{eff}}{\mu_0 M_S}$ quantifying the coupling between the magnetic moment and the easy axis. In Eq. 1.10, the first

term represents the torque exerted by the moment–easy axis coupling. Thermal noise is introduced through the stochastic terms $\vec{H}_{th}$ and $\vec{\theta}_{th}$, defined by:

$$\langle H^i_{th}(t)\rangle = 0, \tag{1.11}$$

$$\langle H^i_{th}(t)H^j_{th}(t')\rangle = \frac{2k_BT}{\mu_0\mu\gamma}\frac{\alpha}{1+\alpha^2}\delta_{ij}\delta(t-t'), \tag{1.12}$$

$$\langle \theta^i_{th}(t)\rangle = 0, \tag{1.13}$$

$$\langle \theta^i_{th}(t)\theta^j_{th}(t')\rangle = 2k_BT\zeta\delta_{ij}\delta(t-t'). \tag{1.14}$$

These relations specify the statistical properties of $\vec{H}_{th}$ and $\vec{\theta}_{th}$, which originate from numerous random microscopic degrees of freedom affecting the orientation of both the magnetic moment and the particle. They are modeled with zero mean, consistent with spatial isotropy. The Kronecker delta δ_{ij} ensures the absence of spatial correlations among thermal components, while the Dirac delta $\delta(t-t')$ reflects the assumption that temporal correlations are negligible compared to the system's response time. Altogether, these noise terms are treated as white noise processes that incorporate the relevant parameters of the magnetic fluid.

The system dynamics are obtained by numerically integrating $2N$ stochastic, coupled vector differential equations (Sec. 2). This approach follows the model proposed by Zapata and Restrepo (2013) [2]; it is also supported by earlier studies by Reeves and Weaver (2015) [3] and Shasha and Krishnan (2020) [4].

1.4. Viscosity

Since the nanoparticles are suspended in a solvent whose viscosity strongly depends on temperature, a consistent model must be adopted. The viscosity–temperature relation must account for two regimes: a solid state, where the concept of viscosity is meaningless (no particle mobility), and a liquid state, where viscosity decreases exponentially with increasing temperature (H. Vogel, 1921) [5]. Accordingly, the following expression is proposed:

$$\eta(T) = \begin{cases} +\infty & , T < T_M \\ 10^{-3}\exp\left(A + \frac{B}{T-C}\right) & , T_M \le T \le T_{max} \end{cases}, \tag{1.15}$$

with T_M the solvent melting temperature and A, B, C, and T_{max} are solvent-specific parameters as prescribed by the Vogel–Fulcher–Tammann (VFT) model [5, 6, 7]. Fig. 1.2 shows the general behavior for water [2, 8].

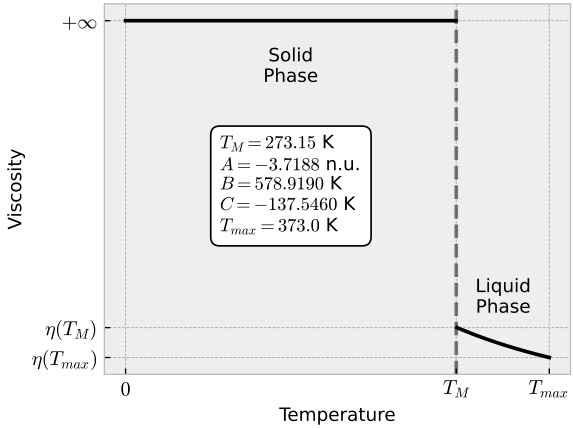


Figure 1.2. Temperature dependence of solvent viscosity.

CHAPTER 2

Numerical Modeling

2.1. Introduction

In this chapter, the numerical implementation of the physical model introduced in Chapter 1 is presented. The numerical formulation combines Langevin-type equations with a Stratonovich-consistent integration scheme to accurately represent deterministic and stochastic effects, providing the computational foundation for the stochastic modeling approach.

Section 2.2 reformulates the equations of motion into a set of *Langevin equations*, separating the deterministic and stochastic components. This representation allows the system to be expressed in terms of vector and matrix operators that couple magnetic and rotational degrees of freedom under thermal noise.

Section 2.3 describes the *Heun–Stratonovich algorithm* employed for the numerical integration of the Langevin equations. The method combines predictor–corrector steps to achieve second-order convergence and ensures a physically consistent stochastic interpretation with experimental observations.

Section 2.4 discusses how *thermal fluctuations* are treated numerically. Thermal noise is efficiently implemented using the Box-Muller algorithm, where stochastic terms are represented as Wiener processes with Gaussian statistical properties.

Section 2.5 defines the main *physical observables* derived from the model, including volumetric magnetization, magnetization vs magnetic field, remanent magnetization, coercive field, specific loss power, and other quantities such as magnetization vs temperature, anisotropy barrier distribution, blocking temperature, some of which are planned for future implementation.

2.2. Langevin Equations

By suitably manipulating the equations of motion 1.7 and 1.8, is obtained:

$$\frac{d\hat{m}}{dt} = \vec{A}(\hat{m}, \hat{n}, t) + \mathbb{B}(\hat{m}, t)\vec{H}_{th}, \tag{2.1}$$

$$\frac{d\hat{n}}{dt} = \vec{C}(\hat{m}, \hat{n}, t) + \mathbb{D}(\hat{n}, t)\vec{\theta}_{th}, \tag{2.2}$$

with $\vec{A}$ and $\vec{C}$ vector operators, and $\mathbb{B}$ and $\mathbb{D}$ matrix operators, defined as:

$$\vec{A} = -\frac{\gamma}{1+\alpha^2}\left[\hat{m} \times \vec{H}_{int} + \alpha\ \hat{m} \times (\hat{m} \times \vec{H}_{int})\right], \tag{2.3}$$

$$\mathbb{B} = -\frac{\gamma}{1+\alpha^2}\left[\hat{m} \times + \alpha\ \hat{m} \times (\hat{m}\times)\right], \tag{2.4}$$

$$\vec{C} = -\frac{1}{\zeta}\ \hat{n} \times \vec{\theta}_{int}, \tag{2.5}$$

$$\mathbb{D} = -\frac{1}{\zeta}\ \hat{n}\times, \tag{2.6}$$

with the interaction field and torque given by:

$$\vec{H}_{int} \equiv \vec{H} + H_K(\hat{m} \cdot \hat{n})\hat{n}, \tag{2.7}$$

$$\vec{\theta}_{int} \equiv -2K_{eff}\Omega_m(\hat{m} \cdot \hat{n})(\hat{m} \times \hat{n}). \tag{2.8}$$

In this form, Eqs. 2.1 and 2.2 represent Langevin-type equations, each consisting of a deterministic contribution ($\vec{A}$ and $\vec{C}$) and a stochastic term ($\mathbb{B}$ and $\mathbb{D}$) driven by multiplicative noise sources, $\vec{H}_{th}$ and $\vec{\theta}_{th}$, respectively. These equations can be interpreted in either the Itô or the Stratonovich sense.

2.3. Heun-Stratonovich Algorithm

The **MagFluid3S** framework employs the Heun method (a modified Euler scheme) to integrate the equations of motion 2.1 and 2.2. This choice is motivated by two main reasons: first, the Heun scheme has a higher order of convergence than the Euler method (second vs. first order), making it numerically more stable [9, 10].

Second, the Heun method naturally yields Stratonovich solutions for the stochastic differential equations, whereas the Euler scheme converges to Itô solutions. In magnetism, the Stratonovich interpretation is preferred because it allows a straightforward implementation of white noise as thermal fluctuations and produces physically consistent results in agreement with experimental observations [2, 10]. The *MagFluid3S algorithm* follows these steps:

1. Euler Predictor:

$$\hat{m}_E = \hat{m}(t) + \vec{A}(\hat{m}, \hat{n}, t)\Delta t + \mathbb{B}(\hat{m}, t)\vec{W}_H, \tag{2.9}$$

$$\hat{n}_E = \hat{n}(t) + \vec{C}(\hat{m}, \hat{n}, t)\Delta t + \mathbb{D}(\hat{n}, t)\vec{W}_\theta. \tag{2.10}$$

2. Heun Corrector:

$$\begin{aligned}\hat{m}(t+\Delta t) = \hat{m}(t) &+ \frac{1}{2}\Big[\vec{A}(\hat{m}_E, \hat{n}_E, t+\Delta t) + \vec{A}(\hat{m}, \hat{n}, t)\Big]\Delta t \\ &+ \frac{1}{2}\Big[\mathbb{B}(\hat{m}_E, t+\Delta t) + \mathbb{B}(\hat{m}, t)\Big]\vec{W}_H,\end{aligned} \tag{2.11}$$

$$\begin{aligned}\hat{n}(t+\Delta t) = \hat{n}(t) &+ \frac{1}{2}\Big[\vec{C}(\hat{m}_E, \hat{n}_E, t+\Delta t) + \vec{C}(\hat{m}, \hat{n}, t)\Big]\Delta t \\ &+ \frac{1}{2}\Big[\mathbb{D}(\hat{n}_E, t+\Delta t) + \mathbb{D}(\hat{n}, t)\Big]\vec{W}_\theta.\end{aligned} \tag{2.12}$$

3. Normalization:

$$\hat{m} \to \frac{\hat{m}}{|\hat{m}|}, \tag{2.13}$$

$$\hat{n} \to \frac{\hat{n}}{|\hat{n}|}. \tag{2.14}$$

Here, $\vec{W}_H$ and $\vec{W}_\theta$ are Wiener processes associated with the thermal field and torque (Sec. 2.4), and the normalization step improves algorithmic convergence [9, 11]. The integration time step Δt should ideally approach zero to ensure rigorous Wiener processes with continuous-time properties; however, this is numerically infeasible. A practical choice is $\Delta t \sim t_P$, where $t_P = 2\pi/(\gamma H_K)$ is the precession time of the magnetic moment around the anisotropy field, representing the smallest time scale in the system. This choice ensures stability, keeping $\Delta\hat{m}, \Delta\hat{n} \lesssim 10^{-2}$ [2, 10].

2.4. Thermal Fluctuations

Applying the proposed integration, the stochastic terms are transformed as:

$$\vec{H}_{th} \to \vec{W}_H \equiv \int_t^{t+\Delta t} d\tau\, \vec{H}_{th}(\tau), \tag{2.15}$$

$$\vec{\theta}_{th} \to \vec{W}_\theta \equiv \int_t^{t+\Delta t} d\tau\, \vec{\theta}_{th}(\tau), \tag{2.16}$$

integrals cannot be computed directly, but their statistical properties are readily obtained (Apps. A.2 and A.3):

$$\langle W_H^i \rangle = 0, \tag{2.17}$$

$$\langle W_H^i W_H^j \rangle = \sigma_H^2\, \delta_{ij}, \tag{2.18}$$

$$\langle W_\theta^i \rangle = 0, \tag{2.19}$$

$$\langle W_\theta^i W_\theta^j \rangle = \sigma_\theta^2\, \delta_{ij}, \tag{2.20}$$

with σ_H and σ_θ quantify the strength of thermal fluctuations. These expressions correspond to Wiener processes, which are characterized by zero mean, continuous trajectories, and successive, stationary, independent increments of size Δt. The Wiener process follows a Gaussian probability distribution and can be efficiently implemented using the *Box–Muller algorithm* [12].

2.5. Physical Observables

Volumetric Magnetization ($\vec{M}$):

The volumetric magnetization, $\vec{M} = \vec{M}(t, H, T)$, is given by the vector sum of the N magnetic moments $\vec{\mu}_i$ at a given time, magnetic field, and temperature:

$$\vec{M}(t, H, T) = \frac{1}{V_m} \sum_{i=1}^{N} \vec{\mu}_i(t, H, T), \tag{2.21}$$

with $V_m = \sum_{i=1}^{N} \Omega_m$ the total volume occupied by the magnetic cores.

Magnetization vs Magnetic Field ($MvsH$):

The volumetric magnetization (Eq. 2.21) is computed under a time-dependent oscillating magnetic field of frequency f at a constant temperature:

$$\vec{H} = \vec{H}(t; f), \tag{2.22}$$

$$T = T_0, \tag{2.23}$$

producing the M-H curve shown in Fig. 2.1.

Remanent Magnetization (M_R):

The remanent magnetization is determined at the points where the magnetic field changes sign in a $MvsH$ loop, that is, $M_R = M(t, 0, T_0)$. For consecutive data points (H_i, M_i) and (H_{i+1}, M_{i+1}) such that $H_i \cdot H_{i+1} < 0$, it is computed as:

$$M_R = M_i - \left(\frac{M_{i+1} - M_i}{H_{i+1} - H_i} \right) H_i. \tag{2.24}$$

The following conditions apply (see Fig. 2.1):

- Upper Branch: M_R^{u} is obtained when $H_i > 0$ and $H_{i+1} < 0$.
- Lower Branch: M_R^{d} is obtained when $H_i < 0$ and $H_{i+1} > 0$.

Coercive Field (H_C):

The coercive field is determined at the points where the volumetric magnetization changes sign in a $MvsH$ loop, that is, $0 = M(t, H_C, T_0)$. For consecutive data points (H_i, M_i) and (H_{i+1}, M_{i+1}) such that $M_i \cdot M_{i+1} < 0$, it is computed as:

$$H_C = H_i - \left(\frac{H_{i+1} - H_i}{M_{i+1} - M_i} \right) M_i. \tag{2.25}$$

The following conditions apply (see Fig. 2.1):

- Upper Branch: H_C^{l} is obtained when $M_i > 0$ and $M_{i+1} < 0$.
- Lower Branch: H_C^{r} is obtained when $M_i < 0$ and $M_{i+1} > 0$.

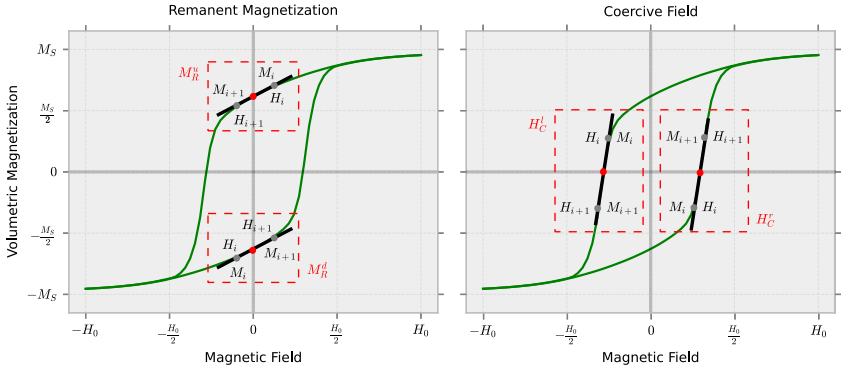


Figure 2.1. *MvsH* loop, remanent magnetization, and coercive field.

Specific Loss Power (*SLP*):

The specific loss power is given by:

$$SLP = \left(\frac{A}{4 M_S H_0} \right) SLP_0, \tag{2.26}$$

$$A = A_{upper} - A_{lower}, \tag{2.27}$$

with SLP_0 the specific loss power constant, M_S the core saturation magnetization, H_0 the magnetic field amplitude, and A the *MvsH* loop area, computed as the difference between the upper and lower branch areas (see Fig. 2.2 and App. A.4).

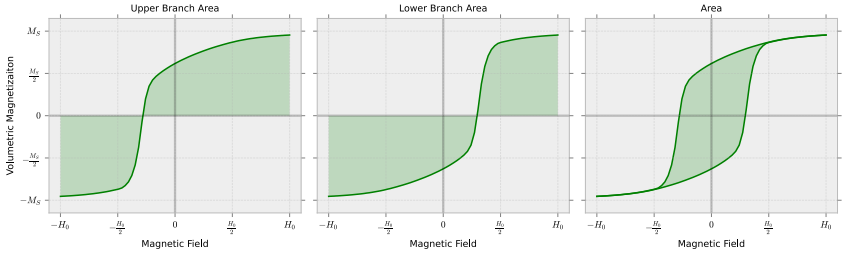


Figure 2.2. Illustration of the *MvsH* loop area determination.

Magnetization vs Temperature ($MvsT$):

The volumetric magnetization (Eq. 2.21) is computed under a temperature ramp from the cold (T_i) to the hot (T_f) reservoir at a constant magnetic field:

$$\vec{H} = H_0 < H_K, \tag{2.28}$$

$$T = T(t), \tag{2.29}$$

producing the M-T curves shown in Fig. 2.3. Two protocols are performed:

- Zero-Field-Cooling (ZFC): cooling at zero magnetic field.
- Field-Cooling (FC): cooling at the saturation magnetic field H_S.

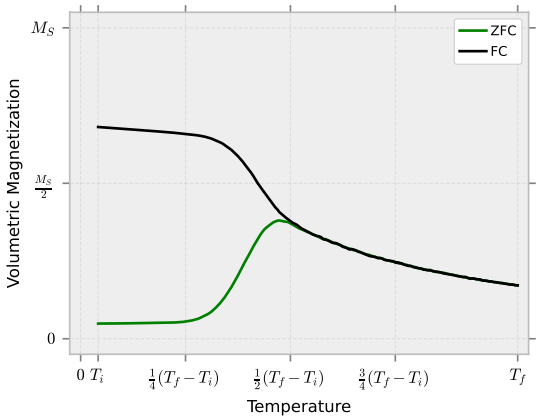


Figure 2.3. ZFC and FC volumetric magnetizations.

Blocking Temperature Distribution (ρ_{T_B}):

The blocking temperature distribution is given by:

$$\Delta M = M_{ZFC} - M_{FC}, \tag{2.30}$$

$$\rho_{T_B} = \frac{d(\Delta M)}{dT}, \tag{2.31}$$

with ΔM the ZFC-FC magnetization difference (see Fig. 2.4).

Blocking Temperature (T_B):

The blocking temperature is calculated as (see Fig. 2.4):

$$T_B = \arg\max(\rho_{T_B}) \,, \tag{2.32}$$

that is, it corresponds to the mode of the blocking temperature distribution.

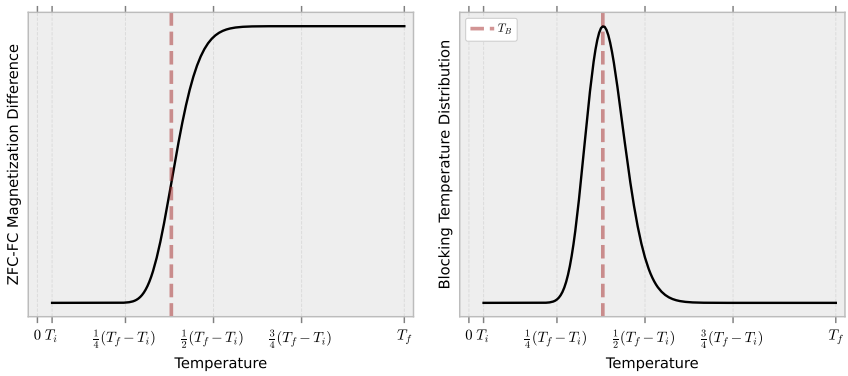


Figure 2.4. Blocking temperature distribution.

Part II

Documentation

CHAPTER 3

Nomenclature

3.1. Introduction

In this chapter, the nomenclature used throughout the **MagFluid3S** framework is presented, providing a comprehensive reference for constants, scalar, vector, and matrix quantities. All quantities are expressed in the International System of Units (SI). The abbreviation 'n.u.' is used throughout to denote dimensionless quantities, while 'a.u.' denote arbitrary units. Depending on their role in different parts of the code, some symbols may appear in scalar, vector, or matrix form.

Section 3.2 details fundamental physical and mathematical *constants*, Section 3.3 defines *scalar* quantities (0D arrays), Section 3.4 introduces *vector* quantities (1D arrays), and Section 3.5 covers *matrix* quantities (2D arrays). These quantities may either be set at the beginning of each simulation, remaining constant, or updated during program execution through data input/output (read/write) operations. Double precision (i.e., float64 in Python or real*8 in Fortran) is used.

Additionally, in the *Summary.txt* files (Chapter 5) or in the source code, certain symbols may be accompanied by the notation $< >$ or $\_m$, $\sigma < >$ or $\_s$, and $< > e$ or $\_e$, which denote, respectively, the mean value, the standard deviation, and the margin of error (with 99% confidence) of the corresponding quantity. Likewise, the symbols $\_u$, $\_d$, $\_l$, and $\_r$ indicate the directions up, down, left, and right, respectively.

3.2. Constants

Symbol	Value	Description	Unit
π, PI	3.14159265	Pi	rad
KB	$1.38064900 \times 10^{-23}$	Boltzmann Constant	J/K
$\mu 0$, MU0	$4\pi \times 10^{-7}$	Vacuum Magnetic Permeability	N/A^2
γ, G	$1.76085963 \times 10^{11}\mu 0$	Gyromagnetic Ratio[a]	rad·m/C

Table 3.1. Mathematical and physical constants.

[a] The most commonly used convention is $1.76085963 \times 10^{11}$ rad·Hz/T [3, 4].

In Fortran, the value of π is defined using the expression $4.0 * atan(1.0)$, whereas in Python it is provided by the NumPy library as *np.pi*. As can be seen, both $\mu 0$ and γ depend on its accurate implementation.

3.3. Scalars (0D Arrays)

Symbol	Description	Unit
RM	Core Radius	m
σRM	Standard Deviation of log(RM)	n.u.
δ	Shell Thickness	m
$\sigma\delta$	Standard Deviation of log(δ)	n.u.
RP	Particle Radius	m
ΩM	Core Volume	m^3
ΩP	Particle Volume	m^3
ρM	Core Mass Density	g/cm^3
MS	Core Saturation Magnetization	A/m
Keff	Effective Anisotropy Constant	J/m^3
HK	Anisotropy Field Amplitude	A/m
α, alp	Damping Parameter[b]	n.u.
θM	Polar Angle (Magnetic Moment)	rad
θN	Polar Angle (Easy Axis)	rad

Table 3.2. Intrinsic parameters.

Symbol	Description	Unit
N	Number of Particles	n.u.
T0, T_	Temperature[c]	K
Ti	Initial Temperature	K
Tf	Final Temperature	K
H0	Magnetic Field Amplitude	A/m
HS	Saturation Field Amplitude	A/m

Table 3.3. External parameters.

Symbol	Description	Unit
tp	Precession Time[a]	s
dt	Integration Time	s
tt	Total Time	s
t	Current Time	s
X0	Number of Loops	n.u.
X1	Curve Points	n.u.
X2	Integration Steps	n.u.
f	Magnetic Field Oscillation Frequency	Hz

Table 3.4. Time parameters.

Symbol	Description	Unit
i, j	Particle Iterator	n.u.
k	File Iterator	n.u.
k0	Loop Iterator	n.u.
k1	Curve Points Iterator	n.u.
k2	Integration Steps Iterator	n.u.
l1, l2, l3	List Iterator	n.u.

Table 3.5. Iteration indices.

Symbol	Description	Unit	Expression
θ	Random/Oriented Polar Angle	rad	$U(0,\pi)$ or θ
ϕ	Random Azimuthal Angle	rad	$U(0,2\pi)$
Ca	LLG Precession Constant	m/C	$\frac{\gamma}{1+\alpha^2}$
Cb	LLG Damping Constant	m/C	$\frac{\gamma\alpha}{1+\alpha^2}$
n	Number of Experiments	n.u.	Not applicable
m	Slope (MvsH Curve, M_R or H_C)	n.u.	$(M_{i+1}-M_i)/(H_{i+1}-H_i)$

Table 3.6. Auxiliary parameters and variables.

Symbol	Description	Unit
TM	Melting Temperature	K
T_MAX	Maximum Temperature	K
eta, ETA_	Solvent Viscosity[c]	Pa·s
DH	Thermal Field Diffusion Coefficient	A^2s/m^2
D0	Thermal Torque Diffusion Coefficient	J^2s
SH	Thermal Field Standard Deviation	As/m
S0	Thermal Torque Standard Deviation	Js
Mui	*i*th-Magnetic Moment (Magnitude)	Am^2
Zi	*i*th-Drag Coefficient	Js
SHi	*i*th-Thermal Field Standard Deviation	As/m
S0i	*i*th-Thermal Torque Standard Deviation	Js

Table 3.7. Scalar thermal, transport, and magnetic quantities.

Symbol	Description	Unit
Vm	Total Core Volume	m^3
MR	Remanent Magnetization	A/m
HC	Coercive Field	A/m
A_u	MvsH Loop Area (Upper Branch)	A^2/m^2
A_l	MvsH Loop Area (Lower Branch)	A^2/m^2
A	MvsH Loop Area	A^2/m^2
SLP0	Specific Loss Power Constant	W/mg
SLP	Specific Loss Power	W/mg
TB	Blocking Temperature	K

Table 3.8. Scalar physical observables.

[a] Will be implemented in a future version.
[b] Alternative definition used in Fortran due to lack of Unicode support.
[c] Variables with hyphens refer to functions defined in Fortran.

3.4. Vectors (1D Arrays)

Symbol	Description	Size	Unit
Rm	Core Radii	N	m
Rp	Particle Radii	N	m
Ω	Core/Particle Volumes	N	m^3
Ωm, Om	Core Volumes	N	m^3
Ωp, Op	Particle Volumes	N	m^3
μ, Mu	Magnetic Moments (Magnitude)	N	Am^2
Emi, Emi_e	ith-Magnetic Moment (Vector)[b]	3	n.u.
Eni, Eni_e	ith-Easy Axis[b]	3	n.u.

Table 3.9. Core and particle properties.

Symbol	Description	Size	Unit
Z, Z_	Drag Coefficients[c]	N	Js
SH, SH_	Thermal Field Standard Deviations[c]	N	As/m
S0, S0_	Thermal Torque Standard Deviations[c]	N	Js

Table 3.10. Vector transport and thermal quantities.

Symbol	Description	Size	Unit
H, H_	Magnetic Field[c]	3	A/m
Ha	Time-Dependent Magnetic Field (H=H(t))	3	A/m
Hb	Time-Dependent Magnetic Field (H=H(t+dt))	3	A/m
HS_ZFC	ZFC Saturation Field	3	A/m
HS_FC	FC Saturation Field	3	A/m

Table 3.11. Magnetic field.

Symbol	Description	Size	Unit	Expression
WH	Wiener Thermal Field	3	As/m	$\vec{W}_H$
W0	Wiener Thermal Torque	3	Js	W_θ
Hint	Interaction Magnetic Field	3	A/m	$\vec{H}_{int}$
Oint	Interaction Magnetic Torque	3	J	$\vec{\theta}_{int}$
A, A_e	LLG Deterministic Term[b]	3	1/s	$\vec{A}(\hat{m}, \hat{n}, t)$
BWH, BWH_e	LLG Stochastic Term[b]	3	n.u.	$B(\hat{m}, t)\vec{W}_H$
C, C_e	LLG-T Deterministic Term[b]	3	1/s	$\vec{C}(\hat{m}, \hat{n}, t)$
DW0, DW0_e	LLG-T Stochastic Term[b]	3	n.u.	$D(\hat{n}, t)\vec{W}_\theta$

Table 3.12. Stratonovich-Heun terms.

Symbol	Description	Size	Unit
t	Time	X2, X1 or X0·X1	s
T	Temperature	X1	K
H	Magnetic Field (z-Direction)	X0·X1	A/m
M	Volumetric Magnetization (z-Direction)	X0·X1	A/m
M_ZFC	ZFC Volumetric Magnetization (z-Direction)	X1	A/m
M_FC	FC Volumetric Magnetization (z-Direction)	X1	A/m
ΔM	ZFC-FC Magnetization Difference	X1	A/m
ΔM_f	ZFC-FC Magnetization Difference (Fitted)	X1	A/m
ρTB	Blocking Temperature Distribution	X1	A/mK
ρTB_f	Blocking Temperature Distribution (Fitted)	X1	A/mK

Table 3.13. Vector physical observables I.

Symbol	Description	Size	Unit
Rm	Core Radius (Mean)	n	m
σRm	Core Radius (Std. Dev.)	n	m
Rp	Particle Radius (Mean)	n	m
σRp	Particle Radius (Std. Dev.)	n	m
Ωm	Core Volume (Mean)	n	m^3
$\sigma\Omega$m	Core Volume (Std. Dev.)	n	m^3
Ωp	Particle Volume (Mean)	n	m^3
$\sigma\Omega$p	Particle Volume (Std. Dev.)	n	m^3
Vm	Total Core Volume	n	m^3
MR	Remanent Magnetization	n	A/m
HC	Coercive Field	n	A/m
SLP	Specific Loss Power	n	W/mg
TB	Blocking Temperature[a]	n	K

Table 3.14. Vector physical observables II.

[a] Will be implemented in a future version.
[b] The '_e' suffix refers to the Euler predictor (Sec. 2.3).
[c] Variables with hyphens refer to functions defined in Fortran.

3.5. Matrices (2D Arrays)

Symbol	Description	Size	Unit
e	Random/Oriented Unitary Vectors	N,3	n.u.
Em	Magnetic Moments (Vector)	N,3	n.u.
Em_ZFC	ZFC Magnetic Moments (Vector)	N,3	n.u.
Em_FC	FC Magnetic Moments (Vector)	N,3	n.u.
En	Easy Axes	N,3	n.u.
E_ZFC	ZFC Easy Axes	N,3	n.u.
E_FC	FC Easy Axes	N,3	n.u.

Table 3.15. Unitary vectors.

Symbol	Description	Size	Unit
Em1	One-Particle Magnetic Moment (Vector)	X2,3	n.u.
En1	One-Particle Easy Axis	X2,3	n.u.
H	Magnetic Field	X0·X1,3	A/m
M	Volumetric Magnetization	X2,3 or X0·X1,3	A/m
M_ZFC	ZFC Volumetric Magnetization	X1,3	A/m
M_FC	FC Volumetric Magnetization	X1,3	A/m

Table 3.16. Matrix physical observables I.

Symbol	Description	Size	Unit
M	Volumetric Magnetization (z-Direction)	n,X0·X1	A/m

Table 3.17. Matrix physical observables II.

CHAPTER 4

MagFluid3S

4.1. Introduction

In this chapter, a brief description of the **MagFluid3S** framework is provided, outlining its structure, workflow, and main components. This architecture implements two simulation schemes: the *llg* solver, in which only the equation of motion 1.7 is solved, and the *llg-t* solver, in which the full set of equations 1.7 and 1.8 is solved.

Section 4.2 presents the general workflow, highlighting the sequence of steps for performing a simulation: initialization, execution, data movement, processing, and visualization. Section 4.3 focuses on the *run* file, illustrating how to instantiate a *MagFluid3S* object, select the simulation type and solver, and specify input/output paths. Section 4.4 centers on the *automation* file, showing how to automate simulations given a set of physical properties. Section 4.5 summarizes the main classes and their associated attributes and methods, and Section 4.6 describes the internal modules, methods, and programs that implement the workflow, providing the numerical routines, utility functions, and algorithms that underpin the framework.

4.2. Workflow

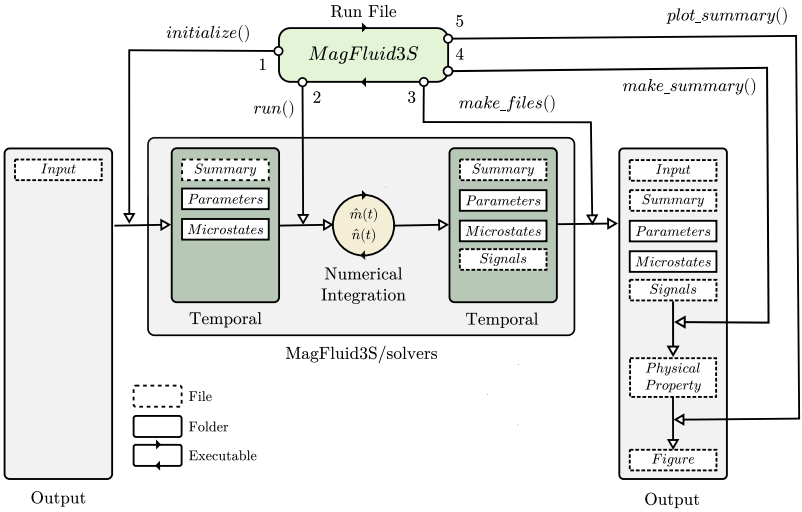


Figure 4.1. MagFluid3S workflow.

A simulation performed using the **MagFluid3S** framework requires three components: a run file (*.py* or *.ipynb*), an input file (*.in*), and an output folder. The workflow proceeds as follows:

1. **Initialization:** The `initialize` method reads the input file and generates, within the *temporal* directory, the simulation parameter files in the *Parameters* folder and the initial condition files in the *Microstates* folder. It also creates the *Summary.txt* file.

2. **Execution:** The `run` method executes the selected simulation (Sec. 5). Numerical integration is carried out using a Fortran-based backend. The resulting data, including thermalization, intermediate, and final states, are stored in the *Microstates* folder. Additionally, a *Signals.txt* file, which contains time, magnetic field, and temperature data, is generated.

3. **Data Moving:** The `make_files` method moves the contents of the *temporal* directory to the *output* directory.

4. **Data Processing:** The *make_summary* method processes the simulation output (*Microstates*, *Signals.txt*, and *Summary.txt*) to compute the desired physical properties (Sec. 2.5).

5. **Visualization:** The *plot_summary* method generates graphical representations based on the computed properties and *Summary.txt*.

Steps 1–3 are implemented in the *MagFluid3SBase* class, and steps 4–5 are part of its extended class, *MagFluid3S*. Both the run file and the input file can reside in the same output directory.

4.3. Run File

The *run.ipynb* file provides a concise example of how to execute a simulation using the **MagFluid3S** framework. It follows the workflow shown in Fig. 4.1. The essential content of this file, required for proper execution, is presented below:

```python
from libs.magfluid3s import MagFluid3S
```

```python
sim = MagFluid3S(simulation = 'MvsH',
                 solver = 'llg',
                 input_file = './examples/MvsH/Input.in',
                 output_folder = './examples/MvsH/')
```

```python
sim.initialize()    # Initialization
sim.run()           # Execution
sim.make_files()    # Data Moving
sim.make_summary()  # Data Processing
sim.plot_summary()  # Visualization
```

Script 4.1. Run file structure.

In this Python script, the *MagFluid3S* object is instantiated, and a simulation of type *MvsH* is initialized and solved using the *llg* solver. The input and output paths are explicitly defined using the **`input_file`** and **`output_folder`** parameters. This code can be easily adapted to other scenarios by appropriately modifying the **`simulation`** type (*Microstates*, *MvsH*, or *MvsT*), the **`solver`** version (*llg* or *llg-t*), and the paths.

For a comprehensive understanding, consult the predefined examples (Sec. 5.2).

4.4. Automatization File

The *auto.ipynb* file is an upgrape of the *run.ipynb* file that provides a concise example of how to execute an automated simulation using the **MagFluid3S** framework. The essential content of this file, required for proper execution, is presented below:

```python
from libs.magfluid3s_auto import MagFluid3SAuto
```

```python
auto = MagFluid3SAuto(simulation = 'MvsH',
                      solver = 'llg',
                      input_file = './examples/MvsH/Input.in',
                      output_folder = './examples/Auto_MvsH/',
                      properties = {'RM':6e-9, 'TO':300},
                      n = 5)
```

```python
auto.run()          # Execution
auto.make_summary() # Data Processing
auto.plot_summary() # Visualization
```

Script 4.2. Automatization file structure.

In this Python script, the *MagFluid3SAuto* object is instantiated, and an automated *MvsH*-type simulation is initialized and solved using the *llg* solver. The input and output paths are explicitly defined using the **`input_file`** and **`output_folder`** parameters. Additionally, a dictionary with the physical properties to be simulated is provided as input, and the number of experiments per simulation is set to 5.

This code can be easily adapted to other scenarios by appropriately modifying the **`simulation`** type (*Microstates*, *MvsH*, or *MvsT*), the **`solver`** version (*llg* or *llg-t*), the paths, the available properties (see the inputs files, Sec. 5.2), and **`n`**.

For a comprehensive understanding, consult the advanced examples (Sec. 5.3).

4.5. Classes

The classes, along with their attributes and methods, within the **MagFluid3S** framework that enable its proper execution are briefly described below.

Name	Description
MagFluid3SBase	Initialization and Execution.
MagFluid3S	Data processing and Visualization.
MagFluid3SAuto	Automatization.

Table 4.1. Classes in the **MagFluid3S** framework.

Name	Description	Main Class
simulation	Simulation Type.	MagFluid3SBase
solver	Solver Version.	MagFluid3SBase
input_file	Input File Path.	MagFluid3SBase
output_folder	Output Folder Path.	MagFluid3SBase
temporal_folder	Temporal Folder Path.	MagFluid3SBase
properties	Physical Properties.	MagFluid3SAuto
n	Number of Experiments.	MagFluid3SAuto

Table 4.2. Main attributes in the **MagFluid3S** framework.

Name	Description	Main Class
initialize	Initialization.	MagFluid3SBase
run	Executation.	MagFluid3SBase, MagFluid3SAuto
make_files	Data Moving.	MagFluid3SBase
make_summary	Data Processing.	MagFluid3S, MagFluid3SAuto
plot_summary	Visualization.	MagFluid3S, MagFluid3SAuto

Table 4.3. Main methods in the **MagFluid3S** framework.

4.6. Modules, Methods, and Programs

The modules, methods, and programs within the **MagFluid3S** framework that enable its proper execution are briefly described below. For further details, refer to the documentation included in the source code.

4.6.1. MagFluid3S/libs/auto/

Name	Description
data	Manage summary files by simulation type.
data_Microstates	Will be implemented in a future version.
data_MvsH	Process for the MvsH simulation (Sec. 5.3.2).
data_MvsT	Will be implemented in a future version.

Table 4.4. Methods in the *auto.data* submodule.

Name	Description
plot	Manage visual files by simulation type.
plot_Microstates	Will be implemented in a future version.
plot_MvsH	Plot for the MvsH simulation (Sec. 5.3.2).
plot_MvsT	Will be implemented in a future version.

Table 4.5. Methods in the *auto.plot* submodule.

Name	Description
run	Run an automatization by type and solver version (Sec. 5.3).

Table 4.6. Programs in the *auto.run* submodule.

Name	Description
make_input_file	Make an input file from a template.
folder_zip	Compresses a folder to ZIP.

Table 4.7. Methods in the *auto.utils* submodule.

4.6.2. MagFluid3S/libs/base/

Name	Description
configuration_Rm	Core Radii List (Eq. 1.2).
configuration_Rp	Particle Radii List (Eq. 1.3).
configuration_Ω	Core/Particle Volumes List (Eqs. 1.1 and 1.4).
configuration_μ	Magnetic Moments (Magnitude) List (Eq. 1.1).
configuration_e	Random/Oriented Unitary Vectors List ($\hat{m}$ and $\hat{n}$).

Table 4.8. Methods in the *base.configurations* submodule.

Name	Description
π	Pi.
KB	Boltzmann Constant.
μ0	Vacuum Magnetic Permeability.
γ	Gyromagnetic Ratio.

Table 4.9. Constants in the *base.constants* submodule.

Name	Description
read_parameters	Process the input file (Sec. 5.2).
initialize	Manage initial files by simulation type.
initial_Microstates	Initialize for the Microstates simulation (Sec. 5.2.1).
initial_MvsH	Initialize for the MvsH simulation (Sec. 5.2.2).
initial_MvsT	Initialize for the MvsT simulation (Sec. 5.2.3).

Table 4.10. Methods in the *base.initialize* submodule.

Name	Description
run	Run a simulation by type and solver version (Sec. 4.1).

Table 4.11. Programs in the *base.run* submodule.

Name	Description
mean_std_error	Mean, Standard Deviation, and Margin of Error.
summary_file	Make a summary file (Secs. 4.2 and 5.2).
folder	Create a folder.
clean_folder	Clean a folder.
make_files	Move folder contents to another.

Table 4.12. Methods in the *base.utils* submodule.

4.6.3. MagFluid3S/libs/post/

Name	Description
data	Manage summary files by simulation type.
data_Microstates	Process for the Microstates simulation (Sec. 5.2.1).
data_MvsH	Process for the MvsH simulation (Sec. 5.2.2).
data_MvsT	Process for the MvsT simulation (Sec. 5.2.3).

Table 4.13. Methods in the *post.data* submodule.

Name	Description
plot	Manage visual files by simulation type.
plot_Microstates	Plot for the Microstates simulation (Fig. 5.1).
plot_MvsH	Plot for the MvsH simulation (Fig. 5.2).
plot_MvsT	Plot for the MvsT simulation (Fig. 5.3).

Table 4.14. Methods in the *post.plot* submodule.

Name	Description
si_scale	International System Units Scale.
si_format	International System Units Format.
vol_magnetization	Volumetric Magnetization (Sec. 2.5).
MR_HC	Remanent Magnetization and Coercive Field (Sec. 2.5).
MvsH_area	MvsH Loop Area (Sec. 2.5).
ΔM_ρTB	ZFC-FC Magnetization Difference and Blocking Temperature Distribution (Sec. 2.5).

Table 4.15. Methods in the *post.utils* submodule.

4.6.4. MagFluid3S/libs/

Name	Description
benchmark	Measure performance metrics of a function.

Table 4.16. Methods in the *decorators* module.

4.6.5. MagFluid3S/solvers/libs/

Name	Description
PI	Pi.
KB	Boltzmann Constant.
MU0	Vacuum Magnetic Permeability.
G	Gyromagnetic Ratio.

Table 4.17. Constants in the *libs.constants* submodule.

Name	Description
r_normal	Random Number (with Normal Distribution).
rv_normal	Random Vector (with Normal Distribution).
dot_prod	Dot Product.
cross_prod	Cross Product.

Table 4.18. Methods in the *libs.math* submodule.

Name	Description
T_	Temperature.
H_	Magnetic Field.
ETA_	Solvent Viscosity (Eq. 1.15).
Z_	Drag Coefficients List (Sec. 1.3).
SH_	Thermal Field Standard Deviations List (App. A.2).
S0_	Thermal Torque Standard Deviations List (App. A.3).

Table 4.19. Methods in the *libs.physics* submodule.

4.6.6. MagFluid3S/solvers/llg/

Name	Description
stratonovich_heun	Perform the S-H algorithm (Eq. 2.11).
evolution	Perform the evolution of the system in one time step.

Table 4.20. Methods in the *llg.integration* submodule.

Name	Description
run_Microstates	Perform the Microstates simulation (Sec. 5.2.1).
run_MvsH	Perform the MvsH simulation (Sec. 5.2.2).
run_MvsT	Perform the MvsT simulation (Sec. 5.2.3).

Table 4.21. Programs in the *llg* module.

4.6.7. MagFluid3S/solvers/llg-t/

Name	Description
stratonovich_heun	Perform the S-H algorithm (Eqs. 2.11 and 2.12).
evolution	Perform the evolution of the system in one time step.

Table 4.22. Methods in the *llg-t.integration* submodule.

Name	Description
run_Microstates	Perform the Microstates simulation (Sec. 5.2.1).
run_MvsH	Perform the MvsH simulation (Sec. 5.2.2).
run_MvsT	Perform the MvsT simulation (Sec. 5.2.3).

Table 4.23. Programs in the *llg-t* module.

CHAPTER 5

How to Run Simulations

5.1. Introduction

In this chapter, the procedure for performing simulations using the **MagFluid3S** framework is described. It provides a structured workflow that connects theoretical models with their computational implementation and serves as a practical guide for executing simulations and adapting workflows to specific projects.

Section 5.2 presents the *Predefined Examples*, which illustrate the three main simulation types: *Microstates*, *MvsH*, and *MvsT*. Each case implements a specific physical protocol: constant-field-and-temperature dynamics, field-dependent magnetization, and temperature-dependent magnetization, respectively. These examples can be executed directly using the *run.ipynb* script, following the guide outlined in Section 4.3.

Section 5.3 presents the *Advanced Examples*, which illustrate automated workflows for the three main simulation types: *Microstates*, *MvsH*, and *MvsT*. These examples focus on the automatization of parameter sweeps and batch simulations, and can be executed directly using the *auto.ipynb* script, following the guide outlined in Section 4.4.

5.2. Predefined Examples

The *examples* folder contains predefined cases that illustrate the types of simulations available in the **MagFluid3S** framework. These examples can be executed directly using the *run.ipynb* file. The predefined simulations are: *Microstates*, *MvsH*, and *MvsT*. Each folder contains an input file (*Input.in*) and output files automatically generated during execution by the Python and Fortran components. Below is a brief description of each simulation type, all of which were performed using the *llg* solver.

5.2.1. Microstates

In this simulation, the fundamental magnetic and structural configuration of a nanoparticle system is studied under constant temperature and magnetic field. It represents the minimal simulation mode in the framework and is used to study the intrinsic dynamics of magnetic moments and easy axes. As such, it serves as a conceptual foundation for more advanced magnetothermal protocols.

Folder Structure:

```
./MagFluid3S/examples/
└── Microstates/
    ├── Microstates/
    │   ├──[2] 0001.txt
    │   ├──[2] 0002.txt
    │   ├── ...
    │   ├──[2] 1000.txt
    │   └──[1] Initial.txt
    ├── Parameters/
    │   ├──[1] External.txt
    │   └──[1] Intrinsic.txt
    ├──[2] Figure.jpg
    ├──[1] Input.in
    ├──[2] M(t;H,T).txt
    ├──[2] One_Particle_Microstates.txt
    ├──[2] Signals.txt
    └──[2] Summary.txt
```

[1] Input file.
[2] Output file.

Folder Description:

Microstates [a b]:

- Contains the system's microstates per time step.

Parameters [a]:

- Contains the system's intrinsic and external parameters.

Figure.jpg [d]:

- Graphical summary of the simulation (see Output subsection).

Input.in:

- Includes system and environment settings (see Input File subsection).

M(t;H,T).txt [c]:

- Time-dependent volumetric magnetization.
- File structure: t [s], M_x [A/m], M_y [A/m], and M_z [A/m].

One_Particle_Microstates.txt [c]:

- Time evolution of $\hat{m}$ and $\hat{n}$ microstates for a random particle.
- File structure: Em_x [n.u.], Em_y [n.u.], Em_z [n.u.], En_x [n.u.], En_y [n.u.], and En_z [n.u.].

Signals.txt [b]:

- Time, magnetic field, and temperature.
- File structure: t [s], H_x [A/m], H_y [A/m], H_z [A/m], and T [K].

Summary.txt [a e]:

- Summary of the simulation and associated statistics.

[a] Generated by the *initialize* method.
[b] Generated by the *run_Microstates* executable.
[c] Generated by the *make_summary* method.
[d] Generated by the *plot_summary* method.
[e] Generated by the *data_Microstates* method.

Input File:

File used to initialize the *Microstates* simulation. Its content is shown below:

```
# Intrinsic Parameters
RM    : 5.0e-9
σRM   : 0.0
δ     : 1.0e-9
σδ    : 0.0
ρM    : 5.240
MS    : 446.0e3
Keff  : 32.0e3
HK    : (2.0*Keff)/(μ0*MS)
α     : 0.1
θM    : -1
θN    : -1

# External Parameters
N     : 1000
T0    : 100.0
H0    : 0.2/μ0

# Time Parameters
dt    : 1.0e-11
X2    : 1000
```

Script 5.1. Microstates input file.

Here, a system of magnetic nanoparticles with a core radius of 5 nm and no size dispersion (σRM = 0), surrounded by a shell of 1 nm thickness (also without dispersion, $\sigma\delta = 0$) is modeled. The material properties of the magnetic phase are determined by the parameters ρM, MS, Keff, HK, and α. The initial orientations of magnetic moments (Em) and easy axes (En) are randomly assigned by setting θM = −1 and θN = −1. A total of N = 1000 nanoparticles are simulated at a temperature of 100 K and under an external magnetic field of 200 mT, which is internally converted to A/m by dividing by μ_0. The system evolves over X2 = 1000 time steps, with an integration time of dt = 100 ns per step.

Output:

The *One_Particle_Microstates.jpg* file shows the time evolution of the magnetic moment (black dots) and the easy axis (red dashed line) of a single randomly nanoparticle. The green arrow represents the direction of the applied magnetic field. The plots on the right display the angle between the magnetic moment and the easy axis ($\theta_{\hat{m},\hat{n}}$), the angle between the magnetic moment and the magnetic field ($\theta_{\hat{m},\vec{H}}$), the angle between the easy axis and the magnetic field ($\theta_{\hat{n},\vec{H}}$), and the reduced volumetric magnetization (M/M_S), all as functions of time.

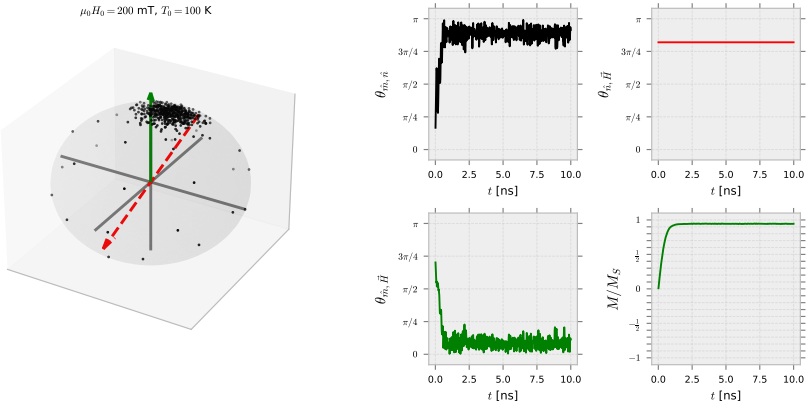


Figure 5.1. Output of the *Microstates* simulation.

5.2.2. MvsH

In this simulation, the magnetic response of a nanoparticle system is analyzed under a constant temperature and a time-dependent cosine external magnetic field given by:

$$\vec{H}(t) = H_0 \cos(2\pi f t)\ \hat{k}, \tag{5.1}$$

with H_0 the magnetic field amplitude, f the oscillation frequency, and t the current time. This advanced mode enables the study of hysteresis phenomena and field-dependent magnetization processes, including coercivity, remanence, and saturation behavior. As such, it serves as a direct extension of the *Microstates* simulation by introducing external field variations as the driving mechanism.

Folder Structure:

```
./MagFluid3S/examples/
└── MvsH/
    ├── Microstates/
    │   ├── 01_001.txt
    │   ├── 01_002.txt
    │   ├── ...
    │   └── 01_200.txt
```

```
├──[1] Initial.txt
└──[1] Saturation.txt
├── Parameters/
│   ├──[1] External.txt
│   └──[1] Intrinsic.txt
├──[2] Figure.jpg
├──[1] Input.in
├──[2] M(t,H;T).txt
├──[2] Signals.txt
└──[2] Summary.txt
```

[1] Input file.
[2] Output file.

Folder Description:

Microstates [a b]:

- Contains the system's microstates per measurement step.

Parameters [a]:

- Contains the system's intrinsic and external parameters.

Figure.jpg [c]:

- Graphical summary of the simulation (see Output subsection).

Input.in:

- Include system and environment settings (see Input File subsection).

M(t,H;T).txt [d]:

- Time-dependent magnetic field and volumetric magnetization.
- File structure: t [s], H_x [A/m], H_y [A/m], H_z [A/m], M_x [A/m], M_y [A/m], and M_z [A/m].

Signals.txt [b]:

- Time, magnetic field, and temperature.
- File structure: t [s], H_x [A/m], H_y [A/m], H_z [A/m], and T [K].

Summary.txt [a e]:

- Summary of the simulation and associated statistics.

[a] Generated by the *initialize* method.
[b] Generated by the *run_MvsH* executable.
[c] Generated by the *plot_summary* method.
[d] Generated by the *make_summary* method.
[e] Generated by the *data_MvsH* method.

Input File:

File used to initialize the *MvsH* simulation. Its content is shown below:

```
# Intrinsic Parameters
RM    : 5.0e-9
σRM   : 0.0
δ     : 1.0e-9
σδ    : 0.0
ρM    : 5.240
MS    : 446.0e3
Keff  : 32.0e3
HK    : (2.0*Keff)/(μ0*MS)
α     : 0.1
θM    : -1
θN    : -1

# External Parameters
N     : 1000
T0    : 100.0
H0    : 0.2/μ0

# Time Parameters
dt    : 1.0e-11
X0    : 1
X1    : 200
X2    : 200
f     : 1.0/(X1*X2*dt)
```

Script 5.2. MvsH input file.

Here, a system of magnetic nanoparticles with a core radius of 5 nm and no size dispersion (σRM = 0), surrounded by a shell of 1 nm thickness (also without dispersion, $\sigma\delta = 0$), is modeled. The material properties of the magnetic phase are determined by the parameters ρM, MS, Keff, HK, and α. The initial orientations of the magnetic moments (Em) and easy axes (En) are randomly assigned by setting θM = −1 and θN = −1. A total of N = 1000 nanoparticles are simulated at a temperature of 100 K and under an external magnetic field of

200 mT, which is internally converted to A/m by dividing by μ_0. An integration time of dt = 100 ns is set, and a single field cycle (X0 = 1) is executed at a frequency of f = 2.5 MHz. The resulting magnetization curve consists of X1 = 200 points (i.e., 200 field steps), and between each of those field values, the system evolves X2 = 200 times.

Output:

The *M(t,H;T).jpg* file shows, on the left, the reduced volumetric magnetization (M/M_S) as a function of the reduced external magnetic field (H/H_0), and on the right, the volumetric magnetization and the external field as functions of time.

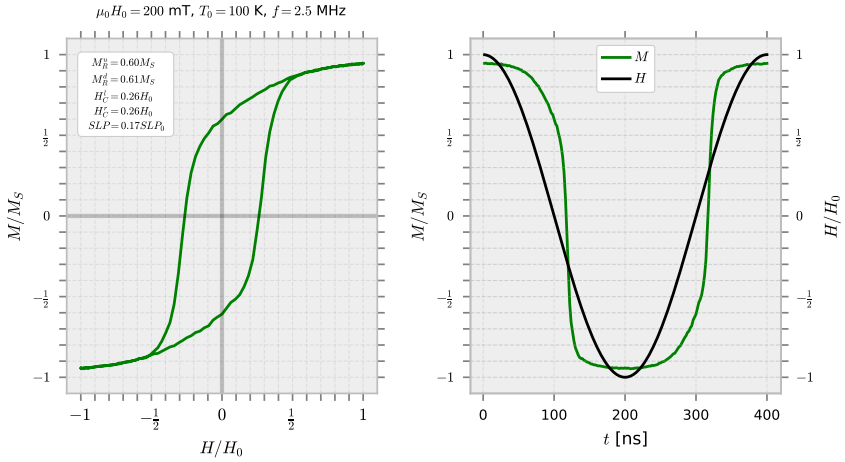


Figure 5.2. Output of the *MvsH* simulation.

5.2.3. MvsT

In this simulation, the thermal response of a magnetic nanoparticle system is analyzed under both Zero Field Cooling (ZFC) and Field Cooling (FC) protocols, with a constant magnetic field and a ramping temperature given by:

$$T(t) = T_i + \left(\frac{T_f - T_i}{t_{total}} \right) t, \tag{5.2}$$

with T_i and T_f the initial and final temperatures, respectively, t_{total} the total simulation time, and t the current time. This advanced mode enables the study of temperature-dependent magnetization, blocking phenomena, and magnetic irreversibility. As such, it serves as a direct extension of the *Microstates* simulation by introducing temperature variations as the driving mechanism.

Folder Structure:

```
./MagFluid3S/examples/
└── MvsH/
    ├── FC Microstates/
    │   ├──[2] 001.txt
    │   ├──[2] 002.txt
    │   ├── ...
    │   ├──[2] 200.txt
    │   ├──[1] Cooling.txt
    │   └──[1] Initial.txt
    ├── Parameters/
    │   ├──[1] External.txt
    │   └──[1] Intrinsic.txt
    ├── ZFC Microstates/
    │   ├──[2] 001.txt
    │   ├──[2] 002.txt
    │   ├── ...
    │   ├──[2] 200.txt
    │   ├──[1] Cooling.txt
    │   └──[1] Initial.txt
    ├──[2] Figure.jpg
    ├──[1] Input.in
    ├──[2] M(t,T;H).txt
    ├──[2] Signals.txt
    ├──[2] Summary.txt
    ├──[2] ΔM(t,T;H).txt
    └──[2] ρTB(t,T;H).txt
```

[1] Input file.
[2] Output file.

Folder Description:

FC Microstates [a b]:

- Contains the system's FC microstates per measurement step.

Parameters [a]:

- Contains the system's intrinsic and external parameters.

ZFC Microstates [a b]:

- Contains the system's ZFC microstates per measurement step.

Figure.jpg [c]:

- Graphical summary of the simulation (see Output subsection).

Input.in:

- Includes system and environment settings (see Input File subsection).

M(t,T;H).txt [d]:

- Time and temperature-dependent volumetric magnetization.
- File structure: t [s], T [K], M_ZFC_x [A/m], M_ZFC_y [A/m], M_ZFC_z [A/m], M_FC_x [A/m], M_FC_y [A/m], and M_FC_z [A/m].

Signals.txt [b]:

- Time, magnetic field, and temperature.
- File structure: t [s], H_x [A/m], H_y [A/m], H_z [A/m], and T [K].

Summary.txt [a e]:

- Summary of the simulation and associated statistics.

ΔM(t,T;H).txt [e]:

- Time, temperature, and ZFC-FC magnetization difference.
- File structure: t [s], T [K], ΔM [A/m], and ΔM_fitted [A/m].

ρTB(t,T;H).txt [e]:

- Time, temperature, and blocking temperature distribution.
- File structure: t [s], T [K], ρTB [A/mK], and ρTB_fitted [A/mK].

[a] Generated by the *initialize* method.
[b] Generated by the *run_MvsT* executable.
[c] Generated by the *make_summary* method.
[d] Generated by the *plot_summary* method.
[e] Generated by the *data_MvsT* method.

Input File:

File used to initialize the *MvsT* simulation. Its content is shown below:

```
# Intrinsic Parameters
RM    : 5.0e-9
σRM   : 0.0
δ     : 1.0e-9
σδ    : 0.0
ρM    : 5.240
MS    : 446.0e3
Keff  : 32.0e3
HK    : (2.0*Keff)/(μ0*MS)
α     : 0.1
θM    : -1
θN    : -1

# External Parameters
N     : 1000
Ti    : 4.0
Tf    : 350.0
HS    : 1.0/μ0
H0    : 5.0e-3/μ0

# Time Parameters
dt    : 1.0e-11
X1    : 200
X2    : 200
```

Script 5.3. MvsT input file.

Here, a system of magnetic nanoparticles with a core radius of 5 nm and no size dispersion (σRM = 0), surrounded by a shell of 1 nm thickness (also without dispersion, $\sigma\delta = 0$), is modeled. The material properties of the magnetic phase are determined by the parameters ρM, MS, Keff, HK, and α. The initial orientations of the magnetic moments (Em) and easy axes (En) are randomly assigned by setting θM = −1 and θN = −1. A total of N = 1000 nanoparticles are simulated from an initial temperature of Ti = 4 K to a final temperature of Tf = 400 K. In the initial conditions, the system is thermalized under a saturation field of HS = 1 T, and during the heating ramp, a cooling field of HC = 5 mT is applied; these are internally converted to A/m by dividing by μ_0. An integration time of dt = 100 ns is set. The resulting magnetization curve consists of X1 = 200

points (i.e., 200 temperature steps), and between each of those temperature values, the system evolves X2 = 200 times.

Output:

The *M(t,T;H).jpg* file shows the reduced volumetric magnetization (M/M_S) as a function of absolute temperature (T) for the system under both Zero Field Cooling (ZFC) and Field Cooling (FC) conditions.

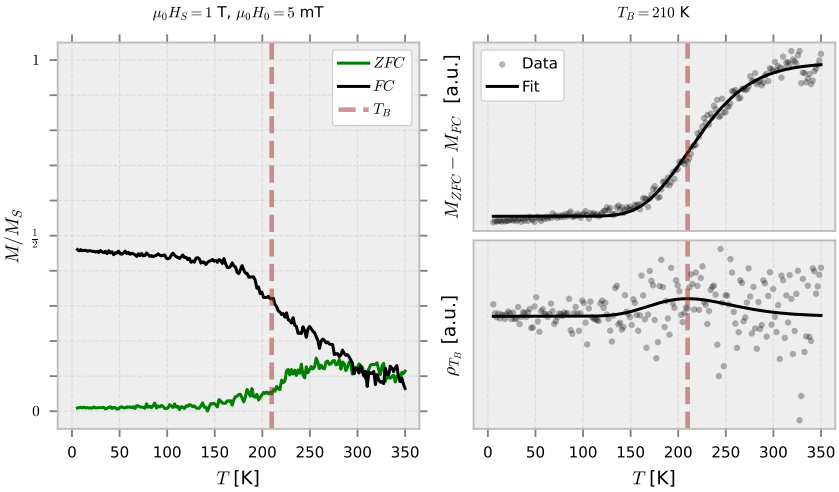


Figure 5.3. Output of the *MvsT* simulation.

5.3. Advanced Examples

Will be implemented in a future version.

5.3.1. Microstates

Will be implemented in a future version.

5.3.2. MvsH

Will be implemented in a future version.

5.3.3. MvsT

Will be implemented in a future version.

Part III

Appendices and References

Appendix

A.1. Release Notes

A.1.1. Version 1.0.0

Release Date: November 23, 2025
Name: Base Version
Tag: v1.0.0

- Initial base version.

A.1.2. Version 1.1.0

Release Date: November 30, 2025
Name: MvsH Upgrades
Tag: v1.1.0

- Minor corrections to the structure and style of the User Guide.
- Added the appendix *Release Notes* to the User Guide.
- Section 1.5, Physical Observables, has been moved to Section 2.5.
- Modifications to the User Guide text (see Secs. 1.1, 1.2, 2.5, 3.1, 3.3, 3.4 and Subsecs. 4.6.2, 4.6.3, 4.6.5, 5.2.1, 5.2.2, and 5.2.3).
- Modifications to the *read_parameters* and *summary_file* methods.
- Added the method *MR_HC* to calculate the remanent magnetization (M_R) and the coercive field (H_C) in the *post.utils* submodule (see Sec. 2.5 and Tab. 4.15).
- Added the method *MvsH_area* to calculate the *MvsH* loop area in the *post.utils* submodule (see Sec. 2.5 and Tab. 4.15).
- Added l1, l2, l3, m, A_u, A_l, and A to Nomenclature section (see Tabs. 3.5, 3.6, and 3.8).

- Added Vm to the *Summary.txt* file (*MvsH* Simulation).
- Modified the *data_MvsH* method to add MR_u, MR_d, HC_l, HC_r, SLP0, and SLP to the *Summary.txt* file.
- Modified the *plot_MvsH* method to add MR_u, MR_d, HC_l, HC_r, and SLP to the *Figure.jpg* file (see Fig. 5.2).
- Updates for consistency and style in the *plot_Microstates*, *plot_MvsH*, and *plot_MvsT* methods (see Figs. 5.1, 5.2, and 5.3).
- Added the appendix *Specific Loss Power* to the User Guide.

A.1.3. Version 1.2.0

Release Date: December 21, 2025
Name: MvsT Upgrades
Tag: v1.2.0

- Modifications to the User Guide text (see Secs. 2.5, 3.1, 3.3, 3.4, 3.5, 4.2, 5.2, Subsec. 4.6.3, and App. A.1.2).
- Modifications to the *initialize_MvsT* and *summary_file* methods.
- Modifications to the *run_MvsT* programs.
- Added the method $\Delta M\_\rho TB$ to calculate the ZFC-FC magnetization difference (ΔM) and blocking temperature distribution (ρ_{T_B}) in the *post.utils* submodule (see Sec. 2.5 and Tab. 4.15).
- Modified the *data_MvsT* method to generate *ΔM(t,T;H).txt* and *ρTB(t,T;H).txt* files.
- Added Vm to the *Summary.txt* file (*Microstates* Simulation).
- Added Vm to the *Summary.txt* file (*MvsT* Simulation).
- Modified the *data_MvsT* method to add TB to the *Summary.txt* file.
- Modified the *plot_MvsT* method to add ΔM, ρTB, and TB to the *Figure.jpg* file (see Fig. 5.3).
- Updates for consistency and style in the *data_Microstates*, *data_MvsH*, and *data_MvsT* methods.

- Updates for consistency and style in the *plot_Microstates*, *plot_MvsH*, and *plot_MvsT* methods (see Figs. 5.1, 5.2, and 5.3).
- Added scikit-learn package as a requirement.

A.1.4. Version 1.3.0

Release Date: January 25, 2026
Name: Automatization I
Tag: v1.3.0

- Modifications to the User Guide text (see Secs. 3.3, 4.1, 4.4, 4.5, 4.6, 5.1 and App. A.1.1, A.1.2, and A.1.3).
- Renamed the method *folder* to *make_folder*.
- Modifications to *MagFluid3SBase* and *MagFluid3S* classes.
- Modifications to *read_parameters* and *summary_file* methods.
- Modifications to *benchmark* method.
- Documentation updates for the methods:
  - *base.utils.make_folder*
  - *base.utils.clean_folder*
  - *post.data.data_Microstates*
  - *post.data.data_MvsH*
  - *post.data.data_MvsT*
  - *post.plot.plot*
  - *post.plot.plot_Microstates*
  - *post.plotplot_MvsH*
  - *post.plotplot_MvsT*
  - *post.utils.MR_HC*
  - *post.utils.MvsH_area*
  - *post.utils.ΔM_ρTB*
- Added the class *MagFluid3SAuto* to the source code.

- Added the section *Automatization File* to the User Guide.
- Added the subsection *MagFluid3S/libs/auto/* to the User Guide.
- Added the subfolder *auto* to *libs*, with the following structure:
  - *__init__.py*
  - *data.py*
  - *plot.py*
  - *utils.py*
- Added the method *make_input_file* to make an input file from a template in the *auto.utils* submodule (see Tab. 4.7).
- Added the method *make_ids* to generate random hexadecimal identifiers in the *auto.utils* submodule (see Tab. 4.7).
- Added the method *folder_zip* to compresses a folder into a ZIP in the *auto.utils* submodule (see Tab. 4.7).
- Added the file *auto.ipynb* to automate the *Microstates*, *MvsH*, and *MvsT* simulations (see Secs. 4.4 and 5.3).
- Added the subsection *Advanced Examples* to the User Guide.
- Added pandas package as a requirement.

A.1.5. Version 1.4.0

Release Date: March 29, 2026
Name: Automatization II
Tag: v1.4.0

- Modifications to the User Guide text (see Frontpage, Secs. 3.1, 3.3, 3.4, 3.5, 4.4, 4.5, 4.6 and 5.2, Subsec. 4.6.1, and A.1.4).
- Modifications to *auto.ipynb* file.
- Modifications to *MagFluid3SAuto* class.
- Modifications to *summary_file*, *mean_std_error*, *benchmark*, *post.data.data_MvsH*, and *post.plot.plot_MvsH* methods.
- Removed the attribute *report* in the *MagFluid3SAuto* class.

- Removed the method *makes_ids*.
- Added the submodule *auto.run* to *libs* (see Tab. 4.6).
- Added the methods *make_summary* and *plot_summary* in the *MagFluid3SAuto* class (see Tab. 4.3).
- Added the methods *data* and *data_MvsH* in the *auto.data* submodule (see Tab. 4.4).
- Added the methods *plot* and *plot_MvsH* in the *auto.plot* submodule (see Tab. 4.5).
- Reorganization of the method parameters in the *MagFluid3SBase* and *MagFluid3S* classes.
- Documentation updates for the methods:
  - *base.initialize.initialize*
  - *base.utils.make_files*
  - *post.data.data*
  - *post.plot.plot*
- Documentation updates for all methods, item *Used by*.
- Improved path portability.
- Image saving changed from JPEG to PDF.

A.2. Thermal Field as a Wiener Process

Based on the definition given in Eq. 2.15 and the rules imposed on the thermal field in Eqs. 1.11 and 1.12, it is shown that:

$$\begin{aligned}
\langle H_{th}^{i}(\tau)\rangle &= 0 \\
\int_{t}^{t+\Delta t} d\tau \, \langle H_{th}^{i}(\tau)\rangle &= 0 \\
\left\langle \int_{t}^{t+\Delta t} d\tau \, H_{th}^{i}(\tau) \right\rangle &= 0 \\
\langle W_{H}^{i}\rangle &= 0. \qquad \text{(A.3)}
\end{aligned}$$

Additionally:

$$\begin{aligned}
\langle H_{th}^{i}(\tau) H_{th}^{j}(\tau')\rangle &= 2D_H \delta_{ij} \delta(\tau - \tau') \\
\int_{t}^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' \langle H_{th}^{i}(\tau) H_{th}^{j}(\tau')\rangle &= \int_{t}^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' 2D_H \delta_{ij} \delta(\tau - \tau') \\
\left\langle \int_{t}^{t+\Delta t} d\tau H_{th}^{i}(\tau) \int_{t'}^{t'+\Delta t'} d\tau' H_{th}^{j}(\tau') \right\rangle &= 2D_H \delta_{ij} \int_{t}^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' \delta(\tau - \tau') \\
\langle W_{H}^{i} W_{H}^{j}\rangle &= 2D_H \delta_{ij} \int_{t}^{t+\Delta t} d\tau \\
\langle W_{H}^{i} W_{H}^{j}\rangle &= 2D_H \Delta t \delta_{ij} \\
\langle W_{H}^{i} W_{H}^{j}\rangle &= \sigma_H^2 \delta_{ij}, \qquad \text{(A.4)}
\end{aligned}$$

with $\sigma_H = \sqrt{2D_H \Delta t}$ and $D_H = \frac{k_B T}{\mu_0 \mu \gamma} \frac{\alpha}{1+\alpha^2}$, the standard deviation and the thermal field diffusion coefficient, respectively (Tabs. 3.7 and 3.10).

A.3. Thermal Torque as a Wiener Process

Based on the definition given in Eq. 2.16 and the rules imposed on the thermal field in Eqs. 1.13 and 1.14, it is shown that:

$$\begin{aligned}
\langle \theta^i_{th}(\tau) \rangle &= 0 \\
\int_t^{t+\Delta t} d\tau \, \langle \theta^i_{th}(\tau) \rangle &= 0 \\
\left\langle \int_t^{t+\Delta t} d\tau \, \theta^i_{th}(\tau) \right\rangle &= 0 \\
\langle W^i_\theta \rangle &= 0. \qquad \text{(A.5)}
\end{aligned}$$

Additionally:

$$\begin{aligned}
\langle \theta^i_{th}(\tau) \theta^j_{th}(\tau') \rangle &= 2D_\theta \delta_{ij} \delta(\tau - \tau') \\
\int_t^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' \langle \theta^i_{th}(\tau) \theta^j_{th}(\tau') \rangle &= \int_t^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' 2D_\theta \delta_{ij} \delta(\tau - \tau') \\
\left\langle \int_t^{t+\Delta t} d\tau \, \theta^i_{th}(\tau) \int_{t'}^{t'+\Delta t'} d\tau' \, \theta^j_{th}(\tau') \right\rangle &= 2D_\theta \delta_{ij} \int_t^{t+\Delta t} d\tau \int_{t'}^{t'+\Delta t'} d\tau' \delta(\tau - \tau') \\
\langle W^i_\theta W^j_\theta \rangle &= 2D_\theta \delta_{ij} \int_t^{t+\Delta t} d\tau \\
\langle W^i_\theta W^j_\theta \rangle &= 2D_\theta \Delta t \delta_{ij} \\
\langle W^i_\theta W^j_\theta \rangle &= \sigma^2_\theta \delta_{ij}, \qquad \text{(A.6)}
\end{aligned}$$

with $\sigma_\theta = \sqrt{2D_\theta \Delta t}$ and $D_\theta = k_B T \, \zeta$, the standard deviation and the thermal torque diffusion coefficient, respectively (Tabs. 3.7 and 3.10).

A.4. Specific Loss Power

For the hysteresis loop shown in Fig. 2.1, the change in internal energy density is $\Delta U = 0$, and therefore $đQ = -\vec{H} \cdot d\vec{B}$. By performing the integral over the loop and using $\vec{B} = \mu_0(\vec{H}+\vec{M})$, with $\vec{H} \,||\, \vec{M}$, the following expression is obtained:

$$
\begin{aligned}
Q &= -\oint \vec{H} \cdot d\vec{B} \\
&= -\oint \vec{H} \cdot \mu_0(d\vec{H} + d\vec{M}) \\
&= -\mu_0 \left[\oint \vec{H} \cdot d\vec{H} + \oint \vec{H} \cdot d\vec{M} \right] \\
&= -\mu_0 \left[\oint H dH + \oint H dM \right] \\
&= -\mu_0 \left[\cancelto{0}{\frac{H^2}{2}}|_{Loop} + \oint H dM \right] \\
&= -\mu_0 \oint [d(HM) - M dH] \\
&= -\mu_0 \left[\cancelto{0}{HM}|_{Loop} - \oint M dH \right] \\
&= \mu_0 \oint M dH \\
&= \mu_0 \left[\int_{-H_0}^{H_0} M_{lower} dH + \int_{H_0}^{-H_0} M_{upper} dH \right] \\
&= \mu_0 \left[\int_{-H_0}^{H_0} M_{lower} dH - \int_{-H_0}^{H_0} M_{upper} dH \right] \\
&= -\mu_0 \left[\int_{-H_0}^{H_0} M_{upper} dH - \int_{-H_0}^{H_0} M_{lower} dH \right] \\
&= -\mu_0 \, (A_{upper} - A_{lower}) \\
&= -\mu_0 A, \qquad \text{(A.7)}
\end{aligned}
$$

with μ_0 the vacuum magnetic permeability and A the $MvsH$ loop area.

The specific loss power (SLP) or heat that the system dissipates during a magnetization loop, is defined as [2, 4]:

$$\begin{aligned} SLP &\equiv \frac{|Q|}{t_{total}\rho_m} \\ &= \frac{\mu_0 A}{t_{total}\rho_m} \\ &= \frac{\mu_0 f A}{\rho_m} \\ &= \left(\frac{A}{4M_S H_0}\right) SLP_0, \end{aligned} \tag{A.8}$$

with $SLP_0 = \frac{4\mu_0 f M_S H_0}{\rho_m}$ the specific loss power constant; H_0 and f the amplitude and oscillation frequency of the magnetic field; and M_S and ρ_m the saturation magnetization and mass density of the magnetic core.

References

[1] Kralj, S. and Makovec D. Magnetic Assembly of Superparamagnetic Iron Oxide Nanoparticle Clusters into Nanochains and Nanobundles. *ACS Nano* 9, 10, 9700-9707 (2015).

[2] Zapata, J. C. and Restrepo, J. Dynamic hysteretic properties and specific loss power of magnetic nanoparticles in a viscous medium at different thermal baths. *AIP Advances* 13, 105110 (2023).

[3] Reeves, D. B., and Weaver, J. B. Combined Néel and Brown rotational Langevin dynamics in magnetic particle imaging, sensing, and therapy. *Appl. Phys. Lett.* 107, 223106 (2015).

[4] Shasha, C. and Krishnan, K. M. Nonequilibrium Dynamics of Magnetic Nanoparticles with Applications in Biomedicine. *Adv. Mater.* 33, 1904131 (2020).

[5] Vogel H. Das Temperaturabhaengigkeitsgesetz der Viskositaet von Fluessigkeiten. *Phys. Z.* 22, 645 (1921).

[6] Fulcher, G. S. Analysis of recent measurements of the viscosity of glasses. *J. Am. Ceram. Soc.* 8, 339–355 (1925).

[7] Tammann, G. and Hesse, W. Die Abhängigkeit der Viscosität von der Temperatur bie unterkühlten Flüssigkeiten. *Z. Anorg. Allg. Chem.* 156, 245–257 (1926).

[8] DDBST, "Liquid dynamic viscosity", DDBST-VFT-Water, 2025.

[9] García-Palacios, J. L. and Lázaro, F. J. Langevin-dynamics study of the dynamical properties of small magnetic particles. *Phys. Rev. B* 58, 14937 (1998).

[10] Scholz, W., Schrefl, T., and Fidler, J. Micromagnetic simulation of thermally activated switching in fine particles. *J. Magn. Magn. Mater.* 233, 296–304 (2001).

[11] Rogge, H., Erbe, M., Buzug, T. M., and Lüdtke-Buzug, K. Simulation of the magnetization dynamics of diluted ferrofluids in medical applications. *Biomed. Tech.* 58, 601–609 (2013).

[12] Box, G. R. P. and Müller, M. E. A Note on the Generation of Random Normal Deviates. *Ann. Math. Stat.* 29, 610–611 (1958).